

Ciencia de Datos para Politólogos:

Modelos Supervisados

Tomás Olego

2026-04-21

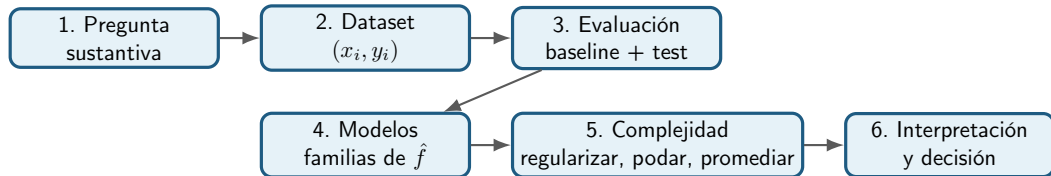
Carrera de Ciencia Política - Universidad de Buenos Aires

Bloque 1: problema supervisado y evaluación

Itinerario lógico de la clase

Idea rectora

La clase no empieza por modelos. Empieza por una decisión: qué queremos predecir, con qué información, cómo mediremos el error y cuándo diremos que el modelo generaliza.



Orden de exposición

Primero fijamos el contrato de evaluación. Luego estudiamos modelos simples e interpretables. Después introducimos mecanismos para ganar flexibilidad sin perder generalización.

Una pregunta social todavía no es un problema supervisado

Pregunta sustantiva

- ¿Qué grupos se perciben como clase obrera?
- ¿Qué variables se asocian con el peso?
- ¿Qué hogares tienen cierta condición de interés?

Problema supervisado

- 1 Definir unidad de análisis.
- 2 Definir resultado Y .
- 3 Definir predictores disponibles X .
- 4 Definir pérdida o métrica.
- 5 Separar datos para evaluación honesta.

Criterio

No alcanza con tener datos. Tiene que quedar claro qué se predice, para quién, con qué información y cómo se juzga el error.

Datos usados como ejemplos guía

Altura-peso

Resultado: weight

Predictores: altura, sexo, edad

Uso: regresión lineal, residuos y R^2 .

Percepción de clase

Resultado: percep_obrera

Predictores: clase, género, edad, educación

Uso: logística, odds y umbrales.

ENUT 2021

Resultado: variable binaria del ejemplo

Predictores: variables sociodemográficas

Uso: CART, bagging y random forest.

Lectura común

Los tres ejemplos tienen la misma estructura: observaciones con predictores x_i y un resultado y_i . Lo que cambia es la naturaleza de Y y la familia de funciones usada para aproximarla.

Problema supervisado: objeto matemático común

Sea un conjunto de entrenamiento

$$\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n,$$

donde $x_i = (x_{i1}, \dots, x_{ip})$ son predictores observados antes de predecir y y_i es el resultado.

Objetivo operativo

Aprender una función $\hat{f}$ que use x para producir una predicción sobre Y en casos nuevos.

Regresión

Y es cuantitativa y el modelo entrega

$$\hat{y}_i = \hat{f}(x_i).$$

Clasificación

Y es cualitativa y el modelo suele entregar

$$\hat{p}_i = \Pr(Y_i = 1 \mid x_i).$$

Qué significa aprender un modelo

Un algoritmo no “descubre la verdad” directamente. Elige una función dentro de una familia posible.

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_{\text{train}}(f), \quad \hat{R}_{\text{train}}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i)).$$

Familia $\mathcal{F}$

Define qué formas puede tener el modelo: rectas, logits, árboles, bosques.

Pérdida ℓ

Define qué errores son caros durante el ajuste: cuadrados, log-loss, impureza, etc.

Punto clave

El entrenamiento minimiza error observado. La evaluación pregunta si esa regla funciona en datos no usados para aprenderla.

Pérdida, métrica y decisión no son lo mismo

Pérdida	Métrica	Decisión
Se usa para ajustar el modelo. • RSS • log-loss • impureza	Se usa para evaluar desempeño. • RMSE, MAE • AUC, F1 • precision, recall	Traduce el score en acción. • umbral • costo de errores • objetivo sustantivo

Regla de clase

Elegir una métrica equivale a declarar qué tipo de error importa más para el problema social concreto.

Baseline: el modelo mínimo que hay que superar

Antes de evaluar modelos complejos conviene definir una referencia simple.

Regresión

Predecir siempre la media de Y estimada en training.

Clasificación

Predecir la clase mayoritaria o la prevalencia estimada en training.

```
from sklearn.dummy import DummyRegressor, DummyClassifier
```

```
dummy_reg = DummyRegressor(strategy="mean")  
dummy_clf = DummyClassifier(strategy="most_frequent")
```

Lectura

Un modelo sofisticado agrega valor sólo si mejora un baseline simple en datos no usados para entrenar.

Predicción, interpretación y causalidad: tres preguntas distintas

Predicción

¿El modelo anticipa bien Y en casos nuevos?

Interpretación

¿Cómo usa el modelo las variables observadas?

Causalidad

¿Qué pasaría con Y si intervinieramos sobre X ?

Advertencia para ciencias sociales

Una asociación predictiva no debe leerse como efecto causal salvo que exista una estrategia de identificación adecuada. La importancia de una variable tampoco es, por sí misma, un efecto causal.

Mapa de modelos de la clase

Familia	Restricción sobre $\hat{f}$	Control de complejidad
Regresión lineal	relación aditiva y lineal	selección de variables, diagnóstico, regularización
Logística	linealidad en log-odds	penalización, umbral, calibración
Ridge/LASSO	lineal, pero con coeficientes encogidos	fuerza de penalización
CART	particiones recursivas del espacio de X	profundidad, hojas mínimas, poda
Bagging/RF	promedio de muchos árboles	número de árboles, decorrelación, hojas

Lectura unificadora

Cada modelo impone una forma distinta de buscar $\hat{f}$. La pregunta de fondo siempre es la misma: cuánta flexibilidad permitir sin ajustar ruido.

Workflow supervisado: evitar contaminar la evaluación

Un modelo predictivo no es sólo una función ajustada. Es una secuencia de decisiones que debe preservar la separación entre aprendizaje y evaluación.



Principio de no filtración

Toda transformación que aprende de los datos debe aprenderse sólo con training, o dentro de cada pliegue de validación cruzada. El test no participa en decisiones.

Train, validación y test: tres roles distintos

Entrenamiento	Validación	Test
Estima parámetros o reglas. $\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_{\text{train}}(f)$	Elige complejidad: $\lambda, C, \text{ccp_alpha}, \text{max_depth},$ $\text{max_features}.$	Estima desempeño fuera de muestra cuando ya no quedan decisiones abiertas.

Regla

Si usamos test para elegir el modelo, deja de ser test y pasa a ser validación.

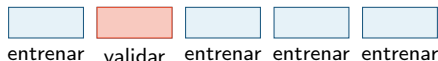
Validación cruzada: elegir complejidad dentro de training

Dividimos el conjunto de entrenamiento en K pliegues. Para cada candidato de complejidad, el modelo se ajusta K veces y se promedia el error de validación:

$$CV_K(\lambda) = \frac{1}{K} \sum_{k=1}^K \hat{R}_{V_k} \left(\hat{f}_{\lambda}^{(-k)} \right).$$

Lectura

La validación cruzada no estima “el mejor coeficiente”. Estima qué nivel de flexibilidad generaliza mejor dentro del conjunto de entrenamiento.



Workflow reproducible en Python

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=123
)

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000))
])

grid = {"model__C": [0.01, 0.1, 1, 10]}
search = GridSearchCV(pipe, grid, cv=5, scoring="roc_auc")
search.fit(X_train, y_train)

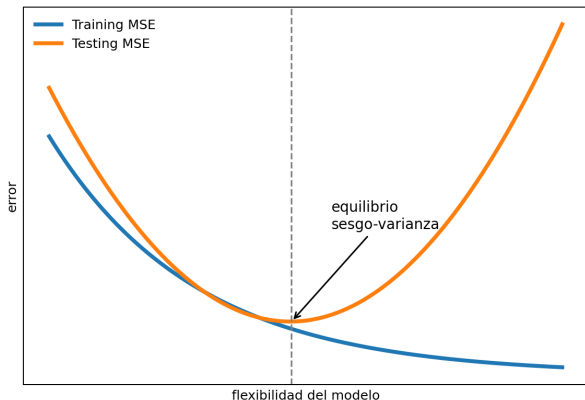
auc_test = search.score(X_test, y_test)
```

Sobreajuste: cuando aprender demasiado empeora generalizar

A medida que aumenta la flexibilidad, el error de entrenamiento suele disminuir. Pero el error de prueba puede subir si el modelo aprende patrones accidentales del training set.

Sobreajuste

Explicar muy bien la muestra observada no equivale a predecir bien casos nuevos.



Cómo leer cada modelo desde ahora

Cuatro preguntas recurrentes

- ① ¿Qué forma puede tener $\hat{f}$?
- ② ¿Qué error intenta minimizar?
- ③ ¿Cómo controla complejidad?
- ④ ¿Cómo se evalúa fuera de muestra?

Transición

Con este contrato de evaluación fijado, el siguiente bloque introduce el modelo más simple e interpretable: la regresión lineal.

Bloque 2: regresión lineal

Regresión lineal: qué se está modelando

La regresión lineal aproxima la esperanza condicional de una variable cuantitativa:

$$\mathbb{E}(Y \mid X = x) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

La restricción fundamental no es que los datos formen una recta perfecta, sino que el cambio medio en Y sea aditivo y lineal en los predictores incluidos.

Parámetro

β_j es una diferencia esperada en Y asociada a una unidad adicional de X_j , manteniendo constantes los demás predictores.

Residuo

$\hat{\varepsilon}_i = y_i - \hat{y}_i$ mide la parte no explicada por el hiperplano ajustado.

Regresión lineal: forma del modelo

La regresión lineal ajusta una recta a datos en los cuales la relación entre X e Y se modela tolerando un error:

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i.$$

Interpretación de parámetros

- β_0 es el intercepto: valor esperado de Y cuando $X = 0$, si el modelo es válido hasta ese punto.
- β_1 es la pendiente: cambio promedio esperado en Y cuando X aumenta una unidad.
- ϵ_i representa la variación no capturada por la recta.

Ejemplo guía: altura y peso

En el ejemplo guía se trabaja con datos de la etnia !Kung San y se restringe el análisis a población adulta. La recta estimada para peso en función de altura es:

$$\widehat{weight}_i = -52.3162 + 0.6294 \, height_i.$$

Predicción

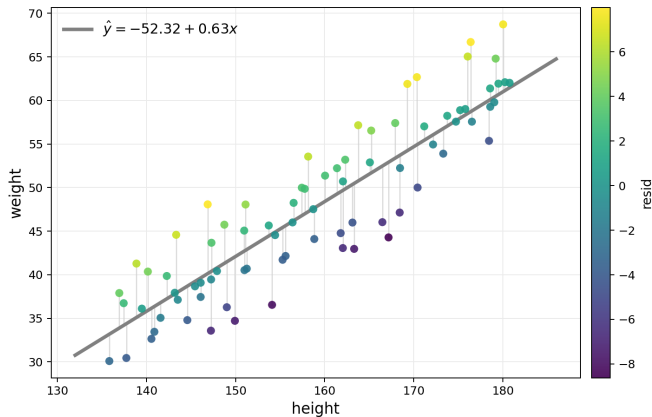
Para una persona de 180 cm:

$$\widehat{weight} = -52.3162 + 0.6294 \cdot 180 \approx 60.98.$$

Lectura sustantiva

Por cada centímetro adicional de altura, se esperan aproximadamente 0.629 kg más de peso, manteniendo la lectura como asociación observacional.

Ajuste visual y residuos



Residuo

$$e_i = y_i - \hat{y}_i.$$

Un punto sobre la recta tiene residuo positivo; un punto bajo la recta tiene residuo negativo.

Ecuación de descomposición

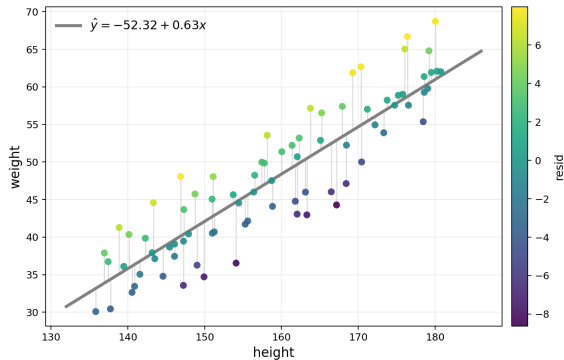
$$y_i = \hat{y}_i + \hat{\varepsilon}_i.$$

Del ajuste a los residuos

El método de mínimos cuadrados ordinarios elige la recta que minimiza la suma de residuos cuadrados:

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Los residuos permiten diagnosticar sesgo sistemático, heterogeneidad de varianza y observaciones influyentes.



Mínimos Cuadrados Ordinarios

El criterio MCO define “la mejor recta” como aquella que minimiza la suma de los residuos cuadrados:

$$RSS(\beta_0, \beta_1) = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Con $\hat{y}_i = \beta_0 + \beta_1 X_i$, se busca:

$$(\hat{\beta}_0, \hat{\beta}_1) = \arg \min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 X_i)^2.$$

Solución para la regresión simple

$$\hat{\beta}_1 = \frac{s_{xy}}{s_x^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{X}.$$

Implementación mínima en Python

```
from sklearn.linear_model import LinearRegression
import pandas as pd

X = df_mayores[["height"]]
y = df_mayores["weight"]

lm_1 = LinearRegression().fit(X, y)
print(lm_1.intercept_, lm_1.coef_[0])

lm_1.predict(pd.DataFrame({"height": [180]}))

df_mayores["weight_predicted"] = lm_1.predict(X)
df_mayores["weight_resid"] = y - df_mayores["weight_predicted"]
```

Sintaxis

`LinearRegression().fit(X, y)`

donde y es el vector respuesta y X es la matriz de predictores, aun en el caso univariado.

R^2 : intuición predictiva

El coeficiente de determinación resume la proporción de variabilidad de Y explicada por el modelo:

$$R^2 = 1 - \frac{SSE}{SST} = \frac{SST - SSE}{SST}.$$

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

En el ejemplo guía, para altura y peso:

$$r = 0.7547, \quad R^2 = 0.5696.$$

El modelo reduce aproximadamente el 57 % de la variación de `weight` al incorporar `height`.

Lectura geométrica de R^2

La variabilidad total de Y se descompone en una parte explicada por el modelo y una parte residual:

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{SST} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{SSR} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{SSE}.$$

Por tanto,

$$R^2 = 1 - \frac{SSE}{SST}.$$

Interpretación

R^2 aumenta cuando la suma de errores cuadrados cae en relación con la variabilidad original de la variable resultado.

R^2 : lectura sustantiva y límites

En regresión lineal, R^2 se interpreta como la proporción de variabilidad de la variable resultado explicada por el modelo:

$$R^2 = \frac{SST - SSE}{SST} = 1 - \frac{SSE}{SST}.$$

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2, \quad SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Lectura sustantiva

Un modelo lineal reduce incertidumbre sobre Y cuando los residuos alrededor de la recta tienen menor dispersión que los valores originales alrededor de $\bar{y}$.

Predictores categóricos: variable indicadora

Para una variable cualitativa con dos niveles se usa una variable indicadora:

$$\widehat{weight}_i = \beta_0 + \beta_1 male_i.$$

En el ejemplo guía:

$$\widehat{weight}_i = 41.81 + 6.778 \times male_i.$$

- β_0 es el promedio estimado del grupo de referencia.
- β_1 es la diferencia promedio entre categorías.
- La interpretación de pendiente e intercepto no cambia: cambia la codificación del predictor.

Regresión lineal múltiple

Al incorporar dos o más variables predictoras:

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \cdots + \beta_p X_{pi} + \epsilon_i.$$

Idea de clase

La regresión múltiple permite considerar simultáneamente variables explicativas y explorar relaciones más complejas entre predictores y respuesta.

Problema práctico

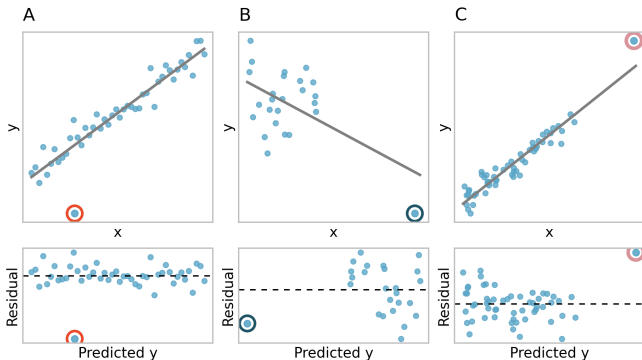
Con muchas variables aparece la pregunta de construcción de modelos: cuáles variables incluir, cómo evaluar colinealidad y cómo balancear ajuste con parsimonia.

Diagnóstico: tres nociones diferentes

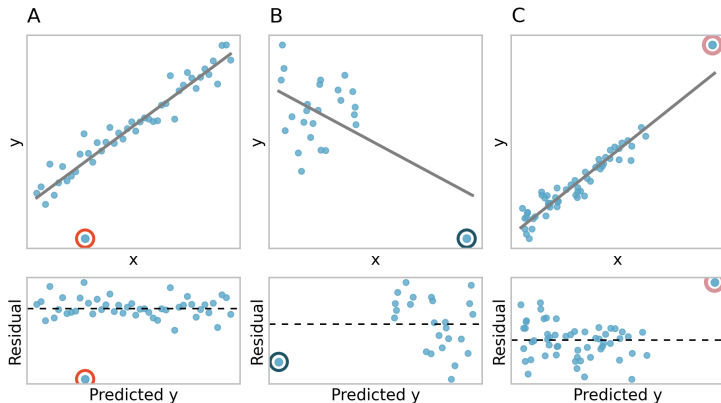
- 1 **Outlier en Y :** residuo grande.
- 2 **Apalancamiento:** valor extremo de X .
- 3 **Influencia:** observación que modifica sustancialmente la recta estimada.

Criterio

Una observación puede tener alto apalancamiento sin ser influyente si cae cerca de la tendencia general.



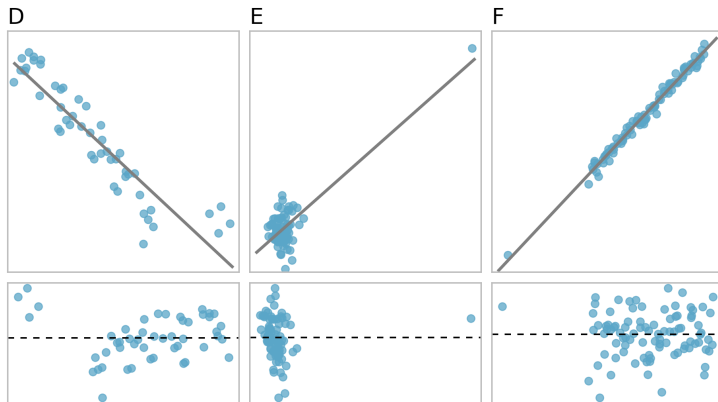
Outliers, apalancamiento e influencia



Lectura

Un valor atípico no siempre es influyente. En regresión lineal importan especialmente los puntos alejados horizontalmente del centro de la nube: puntos de alto apalancamiento.

Outliers: tres patrones adicionales



Advertencia metodológica

Eliminar observaciones atípicas sin justificación sustantiva puede degradar el análisis: los casos excepcionales también pueden ser informativos.

Inferencia en regresión lineal

Bajo el modelo lineal clásico,

$$Y = X\beta + \varepsilon, \quad \mathbb{E}(\varepsilon \mid X) = 0, \quad \text{Var}(\varepsilon \mid X) = \sigma^2 I.$$

El estimador de mínimos cuadrados satisface

$$\hat{\beta} = (X^\top X)^{-1} X^\top Y, \quad \text{Var}(\hat{\beta} \mid X) = \sigma^2 (X^\top X)^{-1}.$$

Idea

La inferencia cuantifica la incertidumbre muestral alrededor de los coeficientes estimados, no sólo el ajuste predictivo.

Intervalos de confianza y tests

Para un coeficiente β_j ,

$$IC_{1-\alpha}(\beta_j) = \hat{\beta}_j \pm t_{1-\alpha/2, n-p} SE(\hat{\beta}_j).$$

El contraste usual

$$H_0 : \beta_j = 0 \quad \text{vs.} \quad H_1 : \beta_j \neq 0$$

usa el estadístico

$$t_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}.$$

Lectura sustantiva

Un intervalo estrecho indica mayor precisión; un intervalo que cruza cero no descarta ausencia de asociación lineal ajustada.

Inferencia lineal en Python

```
import statsmodels.api as sm

X_sm = sm.add_constant(X)
model = sm.OLS(y, X_sm).fit()

print(model.summary())
conf_int = model.conf_int()
coef_table = model.summary2().tables[1]
```

Uso práctico

scikit-learn es conveniente para pipelines predictivos; statsmodels es más directo para errores estándar, intervalos y tests.

Pausa activa 1: leer una regresión

Dada la recta del ejemplo:

$$\widehat{peso} = -52.3 + 0.63 altura$$

- 1 ¿Qué peso predice para una persona de 170 cm?
- 2 ¿Qué significa la pendiente en lenguaje sustantivo?
- 3 ¿Por qué esta pendiente no debe leerse automáticamente como efecto causal?

Objetivo

Separar predicción, interpretación paramétrica y causalidad.

Bloque 3: regresión logística y métricas de clasificación

Motivación: cuando Y es binaria

La regresión logística se introduce para variables de respuesta con dos niveles. En el ejemplo:

$$Y_i = \begin{cases} 1, & \text{percepción de clase obrera,} \\ 0, & \text{no percepción de clase obrera.} \end{cases}$$

Estructura GLM

Los modelos lineales generalizados pueden leerse como un enfoque de dos etapas:

- 1 modelar la variable respuesta mediante una distribución de probabilidad;
- 2 modelar el parámetro de esa distribución con predictores.

Por qué no basta una recta

Si se intentara modelar directamente $p_i = \Pr(Y_i = 1 \mid X_i)$ como una recta,

$$p_i = \beta_0 + \beta_1 X_{1i} + \cdots + \beta_k X_{ki},$$

el lado derecho podría tomar valores menores que 0 o mayores que 1.

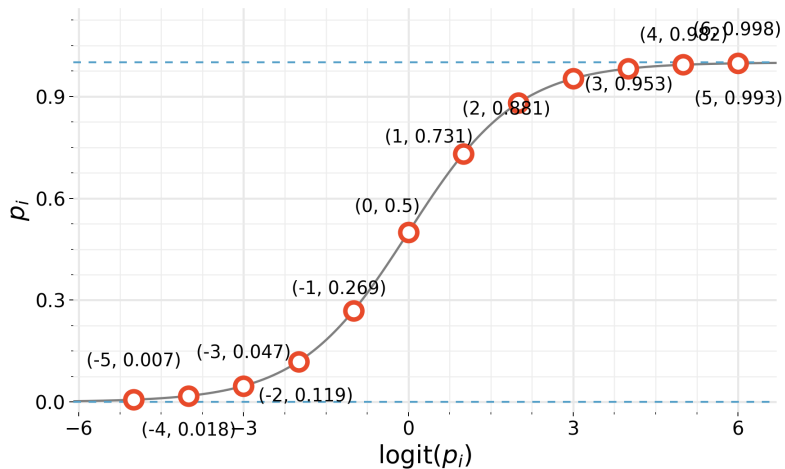
Transformación logit

Para compatibilizar escalas:

$$\text{logit}(p_i) = \log \left(\frac{p_i}{1 - p_i} \right).$$

El logit transforma probabilidades en una escala no acotada.

La transformación logística



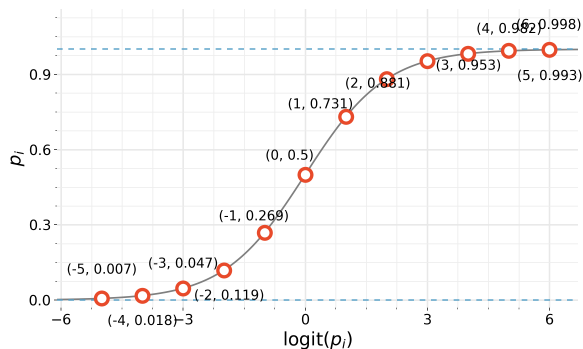
Regresión logística: de una recta a una probabilidad

Si $Y \in \{0, 1\}$, la cantidad natural a modelar es

$$p(x) = \Pr(Y = 1 \mid X = x).$$

La función logística transforma cualquier índice lineal $\eta = x^\top \beta$ en una probabilidad:

$$p(x) = \frac{\exp(\eta)}{1 + \exp(\eta)}.$$



Odds, log-odds y coeficientes

La regresión logística es lineal en los *log-odds*:

$$\log \left(\frac{p(x)}{1 - p(x)} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

Por lo tanto, un incremento de una unidad en x_j multiplica los odds por $\exp(\beta_j)$, manteniendo constantes los demás predictores.

Consecuencia interpretativa

Los coeficientes no son cambios aditivos directos en probabilidad; la variación de probabilidad depende del punto inicial de la curva logística.

Modelo logístico: forma e inversa

El modelo logístico se escribe:

$$\log \left(\frac{p_i}{1 - p_i} \right) = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \cdots + \beta_k X_{ki}.$$

Sea

$$\eta_i = \beta_0 + \beta_1 X_{1i} + \cdots + \beta_k X_{ki}.$$

Entonces la probabilidad predicha es:

$$p_i = \frac{e^{\eta_i}}{1 + e^{\eta_i}} = \frac{1}{1 + e^{-\eta_i}}.$$

Interpretación

Los coeficientes se estiman en escala log-odds; las predicciones suelen comunicarse en escala de probabilidad.

Regresión logística en Python

```
import pandas as pd
import statsmodels.formula.api as smf

df_logit = psh_cony.dropna(
    subset=["percep_obrera", "v111"]
)

glm_1 = smf.logit(
    "percep_obrera ~ C(v111)",
    data=df_logit
).fit()

print(glm_1.summary())
glm_1.predict(pd.DataFrame({"v111": ["Conyuge"]}))
```

Salida del ejemplo

El modelo mínimo estima:

$$\log \left(\frac{P(\text{obrera})_i}{1 - P(\text{obrera})_i} \right) = -0.656 - 0.084 v111_{\text{cony},i}.$$

Modelo logístico con múltiples predictores

El modelo se ajusta con posición de clase, género, posición en el hogar, edad y nivel educativo:

$$\text{percep_obrera} \sim \text{class_eow_agg} + \text{v109} + \text{v111} + \text{v108} + \text{nivel_ed_agg}.$$

Ejemplo de predicción

Para un perfil masculino, PSH, 38 años, educación media y posición de clase propietaria, el ejercicio obtiene:

$$\hat{p}(\text{obrera}) = 0.2498.$$

Advertencia interpretativa

En datos observacionales, las estimaciones pueden cambiar al incorporar o quitar variables del modelo.

Inferencia en regresión logística

En regresión logística se estima β por máxima verosimilitud. Para observaciones binarias,

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)], \quad p_i = \Lambda(x_i^\top \beta).$$

Bajo condiciones regulares,

$$\hat{\beta} \approx N\left(\beta, \mathcal{I}(\hat{\beta})^{-1}\right),$$

donde $\mathcal{I}$ es la información observada.

Lectura

Los errores estándar se interpretan sobre la escala de log-odds; los efectos en probabilidades dependen del punto de evaluación.

Odds ratios: interpretación multiplicativa

Si x_j aumenta una unidad y las demás variables permanecen constantes,

$$\log \left(\frac{p}{1-p} \right) \text{ cambia en } \beta_j.$$

Por lo tanto,

$$OR_j = \exp(\beta_j)$$

es el factor multiplicativo sobre las odds.

Si $OR_j > 1$

aumentan las odds del evento.

Si $OR_j < 1$

disminuyen las odds del evento.

Odds ratio no es cambio constante en probabilidad

Un odds ratio constante no implica un cambio constante en probabilidad.

$$\Delta p = \Lambda(\eta + \beta_j) - \Lambda(\eta)$$

Lectura

El mismo coeficiente puede producir cambios grandes cerca de $p = 0.5$ y cambios pequeños cerca de $p = 0$ o $p = 1$.

Buena práctica

Acompañar odds ratios con probabilidades predichas o efectos marginales en perfiles sustantivamente relevantes.

Odds ratios e intervalos en Python

```
import numpy as np
import statsmodels.api as sm

X_sm = sm.add_constant(X)
logit = sm.Logit(y, X_sm).fit()

coef = logit.params
ci = logit.conf_int()

odds_ratios = np.exp(coef)
odds_ci = np.exp(ci)
```

Advertencia

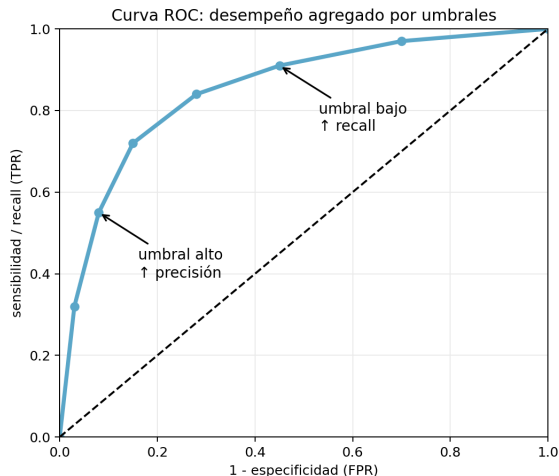
Un odds ratio no es una diferencia de probabilidades. Para comunicar resultados conviene acompañarlo con probabilidades predichas.

Probabilidad estimada y regla de clasificación

Un modelo logístico produce probabilidades. La etiqueta de clase aparece recién al elegir un umbral c :

$$\hat{y}_i(c) = \mathbb{I}\{\hat{p}_i \geq c\}.$$

Mover c cambia sensibilidad, especificidad, precisión y recall. Por eso la evaluación no puede reducirse a un único umbral arbitrario.



Clasificación: umbral y matriz de confusión

La predicción nativa de la regresión logística es una probabilidad. Para clasificar, se aplica una regla:

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i > \tau, \\ 0, & \hat{p}_i \leq \tau. \end{cases}$$

Al variar τ

Cambian accuracy, precision y recall.

Matriz

	$\hat{y} = 1$	$\hat{y} = 0$
$y = 1$	<i>TP</i>	<i>FN</i>
$y = 0$	<i>FP</i>	<i>TN</i>

Clasificación: matriz de confusión

Matriz de confusión y métricas

		Valor real	
		Positivo	Negativo
Predicho	Positivo	TP	FP
	Negativo	FN	TN

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precisión} = \frac{TP}{TP + FP}$$

$$\text{Accuracy} = \frac{TP + TN}{N}$$

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}$$

Pausa activa 2: umbral de clasificación

Un modelo logístico predice $\hat{p} = 0.42$ para una observación.

- 1 ¿Clasifica como 1 si el umbral es 0.5?
- 2 ¿Clasifica como 1 si el umbral es 0.3?
- 3 Si bajamos el umbral, ¿qué tiende a pasar con recall y precision?

Criterio

El umbral no es una propiedad del modelo: es una decisión sustantiva sobre costos de falsos positivos y falsos negativos.

Métricas: regresión y clasificación no miden lo mismo

Regresión

$$MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad RMSE = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$$

$$R^2 = 1 - \frac{RSS}{TSS}.$$

Clasificación

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN},$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}.$$

Regla

Elegir una métrica equivale a definir qué tipo de error es más costoso.

Métricas de clasificación: precisión, recall y F1

Matriz de confusión y métricas

		Valor real	
		Positivo	Negativo
Predicho	Positivo	TP	FP
	Negativo	FN	TN

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precisión} = \frac{TP}{TP + FP}$$

$$\text{Accuracy} = \frac{TP + TN}{N}$$

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}$$

$$F1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

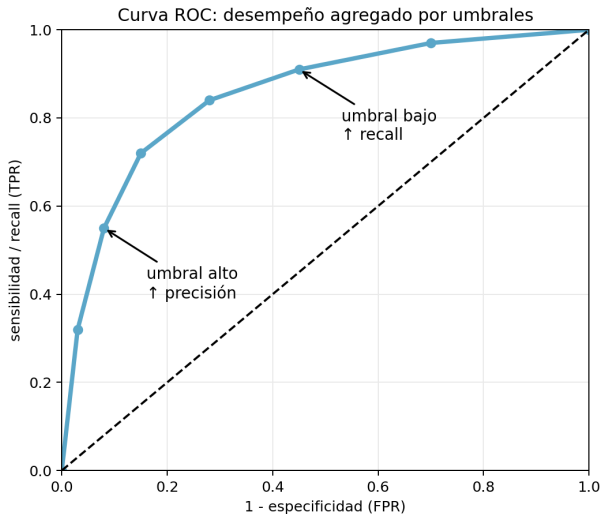
Lectura

- **Precision:** entre los predichos positivos, cuántos eran positivos.
- **Recall:** entre los positivos reales, cuántos se detectaron.
- **F1:** compromiso cuando ambas dimensiones importan.

Métricas en Python

```
from sklearn.metrics import (  
    mean_absolute_error, mean_squared_error, r2_score,  
    accuracy_score, precision_score, recall_score,  
    f1_score, roc_auc_score  
)  
  
rmse = mean_squared_error(y_test, y_pred, squared=False)  
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
  
acc = accuracy_score(y_test, y_class)  
prec = precision_score(y_test, y_class)  
rec = recall_score(y_test, y_class)  
f1 = f1_score(y_test, y_class)  
auc = roc_auc_score(y_test, y_score)
```

ROC y AUC



Curva ROC

Relaciona:

$$TPR = \frac{TP}{TP + FN}$$

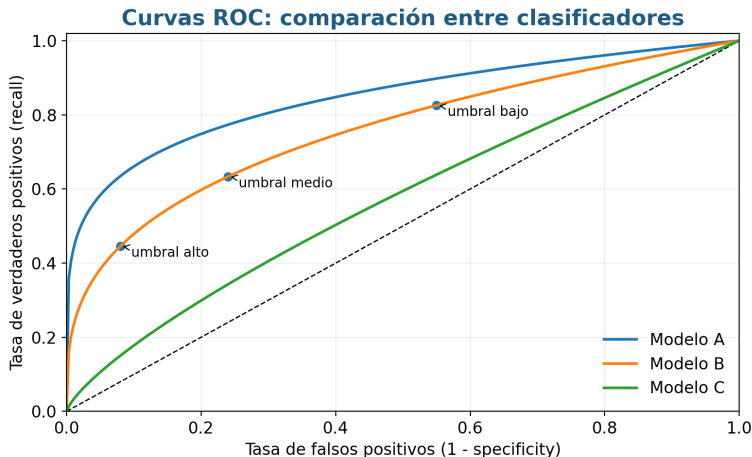
con

$$FPR = 1 - \text{specificity}.$$

AUC

Resume la performance agregada sobre todos los umbrales posibles.

Clasificación: umbrales y curva ROC



Mover el umbral altera simultáneamente recall y falsos positivos. Por eso la curva ROC resume la performance sobre todos los umbrales posibles.

AUC no es calibración

AUC

Mide ranking: si los casos positivos tienden a recibir scores mayores que los negativos.

Calibración

Mide probabilidades: si entre los casos con predicción 0.70, aproximadamente 70 % son positivos.

Un modelo puede tener buen AUC y malas probabilidades calibradas.

```
from sklearn.calibration import CalibrationDisplay  
  
CalibrationDisplay.from_estimator(model, X_test, y_test)
```

Clases desbalanceadas: cuidado con accuracy

Cuando una clase es muy frecuente, accuracy puede ser alta aun con un modelo poco útil.

Métricas a mirar

Precision, recall, F_1 , ROC-AUC y, muchas veces, PR-AUC.

```
from sklearn.metrics import average_precision_score  
  
pr_auc = average_precision_score(y_test, y_score)
```

Regla práctica

Elegir una métrica equivale a decir qué error duele más en el problema sustantivo.

Pausa activa 3: elegir métrica

Caso: en una política pública, los falsos negativos son más costosos que los falsos positivos.

- 1 ¿Qué métrica mirarías primero?
- 2 ¿Conviene subir o bajar el umbral?
- 3 ¿Qué costo interpretativo tiene optimizar sólo esa métrica?

Bloque 4: control de complejidad en modelos paramétricos

Colinealidad: cuando los coeficientes se vuelven inestables

La matriz $X^T X$ resume la información disponible para separar los efectos parciales de los predictores. Si algunos predictores son casi combinaciones lineales de otros, entonces $X^T X$ queda mal condicionada y

$$\text{Var}(\hat{\beta} \mid X) = \sigma^2 (X^T X)^{-1}$$

puede crecer fuertemente.

Consecuencia

Dos modelos con ajuste predictivo similar pueden tener coeficientes muy distintos. Ridge estabiliza esta inversión reemplazando

$$X^T X \quad \text{por} \quad X^T X + \lambda I.$$

Motivación de regularización: colinealidad y varianza

Problema

Predictores correlacionados hacen difícil atribuir una parte única de la variación de Y a cada variable.

Síntoma

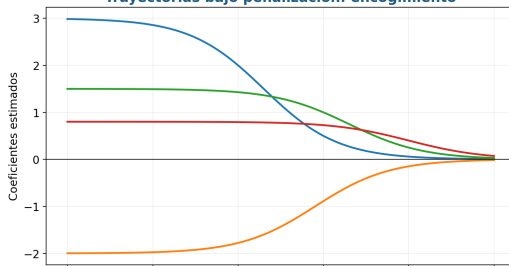
Coefficientes grandes, signos inestables, intervalos amplios.

Respuesta

La penalización agrega sesgo controlado para reducir varianza:

$$\hat{\beta}_{\lambda} = \arg \min_{\beta} \left\{ \sum_i (y_i - x_i^{\top} \beta)^2 + \lambda P(\beta) \right\}.$$

Trayectorias bajo penalización: encogimiento



Control de complejidad: mapa de recursos

Antes de pasar a árboles y ensambles, conviene separar cuatro mecanismos que suelen mezclarse:

Mecanismo	Ejemplo	Qué hace
Regularización	Ridge, LASSO, Elastic Net	Modifica la estimación achicando coeficientes
Selección penalizada	AIC, BIC	Compara modelos ya estimados penalizando cantidad de parámetros
Poda	Cost-complexity pruning	Simplifica árboles eliminando ramas
Validación cruzada	Grid search	Elige hiperparámetros por desempeño fuera de muestra

Orden lógico

En este bloque vemos regularización y selección penalizada para modelos paramétricos. La poda se retoma después, cuando ya esté definida la lógica de CART.

Regularización: estimar con una restricción

La regularización modifica el problema de estimación:

$$\hat{\beta}_{\lambda} = \arg \min_{\beta} \{ \mathcal{L}(\beta) + \lambda J(\beta) \}.$$

El primer término mide ajuste; el segundo penaliza complejidad. El hiperparámetro λ controla el intercambio entre ambos.

Ridge

$J(\beta) = \sum_j \beta_j^2$: contrae coeficientes sin llevarlos exactamente a cero.

LASSO

$J(\beta) = \sum_j |\beta_j|$: puede producir selección automática de variables.

Regresión penalizada: esquema general

La penalización aparece explícitamente al estudiar *cost-complexity pruning*. La forma conceptual es:

$$\text{objetivo penalizado} = \text{pérdida} + \text{penalización}.$$

Lectura para modelos lineales

Para una regresión, la pérdida natural de MCO es:

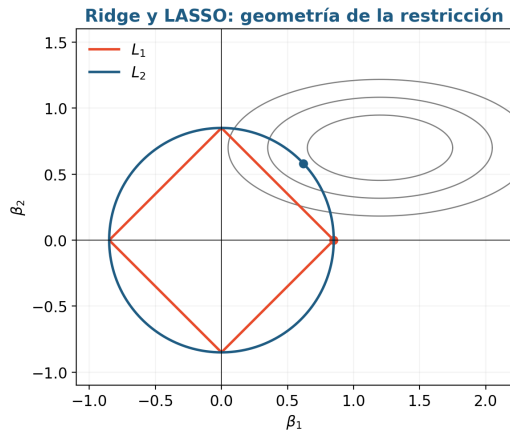
$$\sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Una penalización agrega un costo por complejidad del modelo, controlado por un hiperparámetro.

Punto de enlace

La poda de costo-complejidad puede leerse como una penalización de tipo *Loss + Penalty*, análoga a otras formas de regularización.

Penalización: geometría de Ridge y LASSO



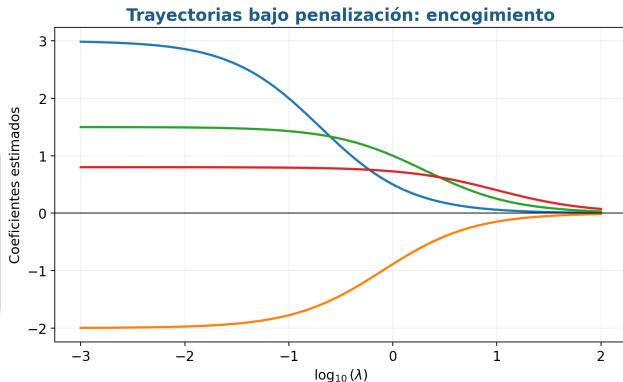
$$\hat{\beta}_{\lambda} = \arg \min_{\beta} \left\{ \sum_{i=1}^n (y_i - x_i^{\top} \beta)^2 + \lambda [(1 - \alpha) \|\beta\|_2^2 + \alpha \|\beta\|_1] \right\}.$$

Trayectorias de regularización

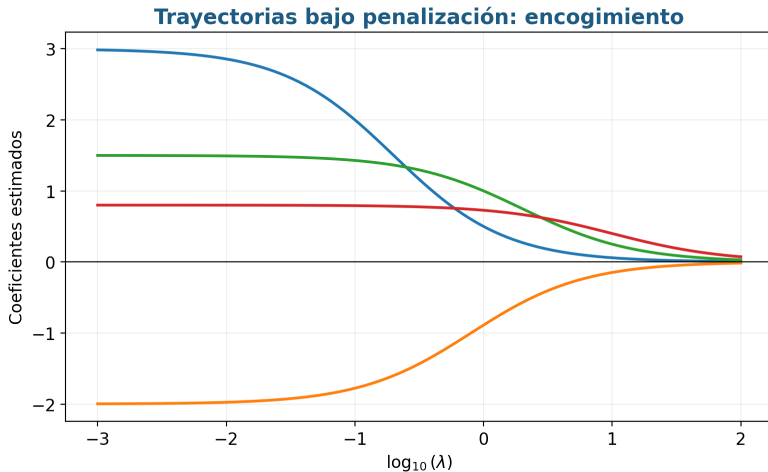
Las trayectorias muestran cómo cambian los coeficientes al variar la intensidad de penalización. Cuando la penalización aumenta, los coeficientes se contraen hacia cero.

Lectura práctica

La validación cruzada elige el nivel de penalización que minimiza error fuera de muestra, no el que maximiza ajuste en entrenamiento.



Penalización: trayectorias de coeficientes



Al aumentar λ , el criterio penalizado contrae los coeficientes. En LASSO, algunos coeficientes pueden quedar exactamente en cero; en Ridge, la contracción suele ser continua.

Selección penalizada: AIC no es shrinkage

En regresión logística, el AIC permite comparar modelos candidatos:

$$\text{AIC} = -2\ell(\hat{\beta}) + 2k,$$

donde k es la cantidad de parámetros estimados.

AIC/BIC

Comparan modelos ya estimados y penalizan cantidad de parámetros.

Ridge/LASSO/Elastic Net

Cambian la estimación: contraen coeficientes durante el ajuste.

Lectura práctica

Menor AIC indica mejor compromiso relativo entre ajuste y parsimonia dentro del conjunto de modelos comparados; no mide desempeño absoluto fuera de muestra.

Regularización en Python: escala y parámetro C

La regularización requiere predictores en escalas comparables; por eso conviene usar `StandardScaler` dentro de un `Pipeline`.

```
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression(
        penalty="elasticnet", solver="saga", l1_ratio=0.5,
        max_iter=5000
    ))
])
```

Atención

En `scikit-learn`, `C` es el inverso de la fuerza de regularización: `C` chico implica más regularización; `C` grande implica menos regularización.

Penalización logística y Elastic Net

La misma idea de regularización se aplica a modelos de clasificación. En lugar de minimizar suma de cuadrados, se minimiza la pérdida logística penalizada:

$$\hat{\beta} = \arg \min_{\beta} \left\{ -\ell(\beta) + \lambda \left[\alpha \|\beta\|_1 + (1 - \alpha) \|\beta\|_2^2 \right] \right\}.$$

Casos particulares

$\alpha = 0 \Rightarrow$ Ridge, $\alpha = 1 \Rightarrow$ LASSO, $0 < \alpha < 1 \Rightarrow$ Elastic Net.

Logística penalizada en Python

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("logit", LogisticRegression(
        penalty="elasticnet",
        solver="saga",
        max_iter=5000
    ))
])

grid = {
    "logit__C": [0.01, 0.1, 1, 10],
    "logit__l1_ratio": [0.0, 0.5, 1.0]
}

search = GridSearchCV(pipe, grid, cv=5, scoring="roc_auc")
search.fit(X_train, y_train)
```

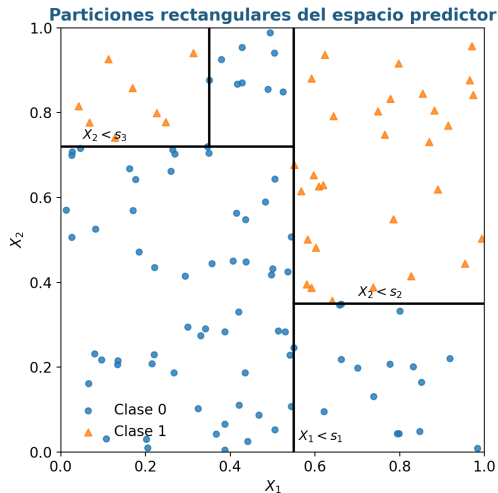
Bloque 5: CART

CART: de predictores a regiones

Un árbol no impone linealidad global. En cambio, divide recursivamente el espacio predictor en regiones rectangulares y asigna una predicción constante dentro de cada región.

Interpretación

Cada camino desde la raíz hasta una hoja es una regla del tipo: si se cumplen ciertas condiciones, entonces predecir una clase o un promedio.



Árboles de decisión: premisa

Los árboles son modelos supervisados para:

regresión de variables numéricas y clasificación de variables categóricas.

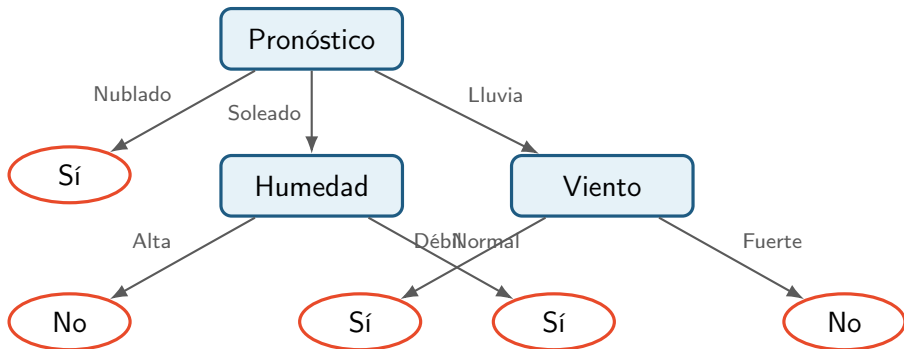
Premisa básica

Dividir el espacio de predictores en regiones más simples, usando reglas de decisión sucesivas.

Propiedad central

Son interpretables y fáciles de explicar, pero esta simplicidad suele venir acompañada de menor capacidad predictiva y baja robustez ante cambios pequeños en los datos.

Árbol de clasificación: ejemplo intuitivo



Idea

Cada nodo aplica una partición y cada hoja devuelve una etiqueta de clase por mayoría.

Ejemplo CART: set de entrenamiento

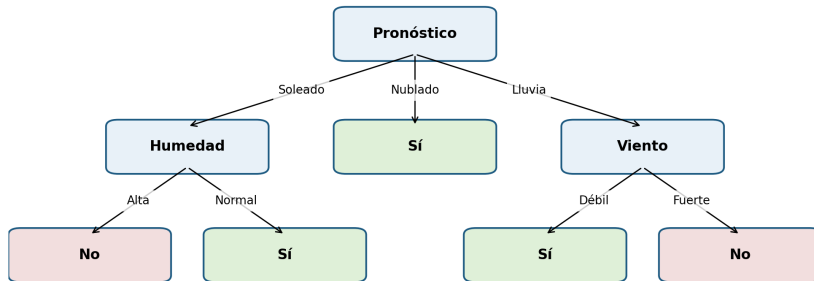
Set de entrenamiento: ¿podemos jugar al fútbol?

Pronóstico	Temperatura	Humedad	Viento	Jugar
Soleado	Alta	Alta	Débil	No
Soleado	Alta	Alta	Fuerte	No
Nublado	Alta	Alta	Débil	Sí
Lluvia	Media	Alta	Débil	Sí
Lluvia	Baja	Normal	Débil	Sí
Lluvia	Baja	Normal	Fuerte	No
Nublado	Baja	Normal	Fuerte	Sí
Soleado	Media	Alta	Débil	No
Soleado	Baja	Normal	Débil	Sí
Lluvia	Media	Normal	Débil	Sí
Soleado	Media	Normal	Fuerte	Sí
Nublado	Media	Alta	Fuerte	Sí
Nublado	Alta	Normal	Débil	Sí
Lluvia	Media	Alta	Fuerte	No

El ejemplo intuitivo construye un árbol para decidir si se puede jugar al fútbol a partir de pronóstico, temperatura, humedad y viento.

Ejemplo CART: árbol resultante

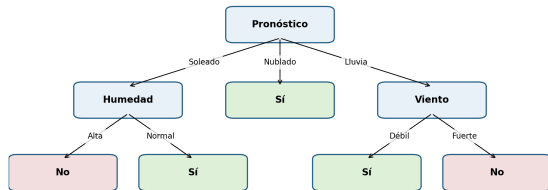
Árbol de clasificación intuitivo



Cada nodo interno aplica un test de atributo; cada hoja devuelve la clase modal dentro de la región inducida por el camino de decisión.

Del árbol conceptual al árbol ajustado

Árbol de clasificación intuitivo



Qué muestra el gráfico

Los nodos internos son preguntas binarias sobre predictores; las hojas son decisiones/predicciones. La estructura se lee de arriba hacia abajo.

Qué se generaliza

En aplicaciones empíricas, el mismo mecanismo se aplica a muchas variables y se tunea mediante profundidad, tamaño mínimo de nodo y poda.

Algoritmo general para árboles de clasificación

Sea D_t el conjunto de entrenamiento que llega al nodo t .

- 1 Si todos los registros en D_t pertenecen a la misma clase y_t , entonces t es hoja rotulada como y_t .
- 2 Si D_t está vacío, entonces t es hoja rotulada por la clase default y_d .
- 3 Si D_t contiene más de una clase, se usa un test de atributo para separar los datos en subconjuntos.
- 4 Se aplica recursivamente el procedimiento a cada subconjunto.

Preguntas de modelado

¿Qué split elegir?

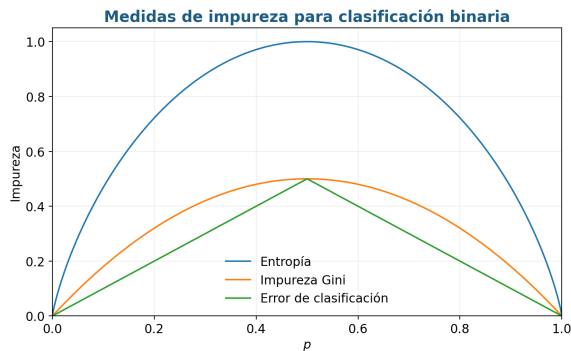
¿Cuándo detener el crecimiento?

Cómo decide un árbol dónde cortar

En clasificación, cada corte candidato se evalúa por la reducción de impureza que produce:

$$\Delta I = I(\text{nodo padre}) - \left(\frac{n_L}{n} I(L) + \frac{n_R}{n} I(R) \right).$$

El algoritmo elige el corte con mayor ganancia local.



Impureza y ganancia

Para elegir entre particiones candidatas se busca reducir la impureza del nodo. Si $p(i | t)$ es la proporción de clase i en el nodo t :

Criterios mencionados

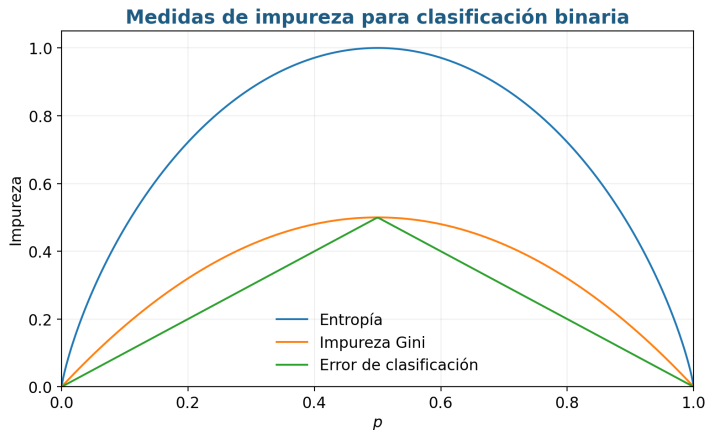
- Classification error rate.
- Índice de Gini.
- Entropía.

Ganancia

Comparar impureza antes y después de dividir:

$$\Delta I = I(\text{padre}) - \sum_j \frac{N_j}{N} I(\text{hijo}_j).$$

Impureza: funciones usadas en clasificación



Para un nodo binario con proporción p de clase positiva:

$$Gini(p) = 2p(1 - p), \quad Entropy(p) = -p \log_2 p - (1 - p) \log_2 (1 - p).$$

Ganancia de impureza

Una partición se elige comparando impureza antes y después del split:

$$\Delta I = I(t) - \sum_{j=1}^J \frac{N_j}{N} I(t_j).$$

- $I(t)$ mide impureza del nodo padre.
- $I(t_j)$ mide impureza del nodo hijo j .
- La ponderación N_j/N evita favorecer splits que sólo mejoran nodos pequeños.

Regla local

CART usa una búsqueda greedy: en cada paso elige el split que maximiza la ganancia local, no necesariamente el árbol globalmente óptimo.

CART: clasificación y regresión comparten una lógica

En ambos casos el árbol busca particiones recursivas del espacio predictor. Cambia la función de pérdida dentro de cada nodo.

Árbol de regresión

Cada región predice una media:

$$\hat{y}_R = \frac{1}{|R|} \sum_{i: x_i \in R} y_i.$$

La impureza se mide con varianza o suma de cuadrados.

Árbol de clasificación

Cada región predice una clase o una probabilidad:

$$\hat{p}_{k,R} = \frac{1}{|R|} \sum_{i: x_i \in R} \mathbf{1}(y_i = k).$$

La impureza se mide con Gini o entropía.

Árboles de regresión

Cuando Y es cuantitativa, la predicción en una región terminal es una media:

$$\hat{y}_i = \bar{y}_{R_j} \quad \text{si} \quad x_i \in R_j.$$

Regiones

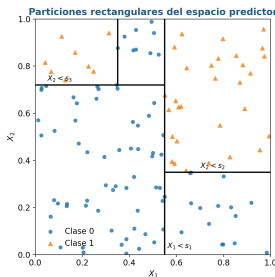
El espacio de predictores se divide en regiones $R_1, \dots, R_J$; para cada observación que cae en R_j , se predice la media de Y en esa región.

Criterio

Se buscan regiones que minimicen la suma de residuos cuadrados:

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^2.$$

Árboles de regresión: particiones rectangulares



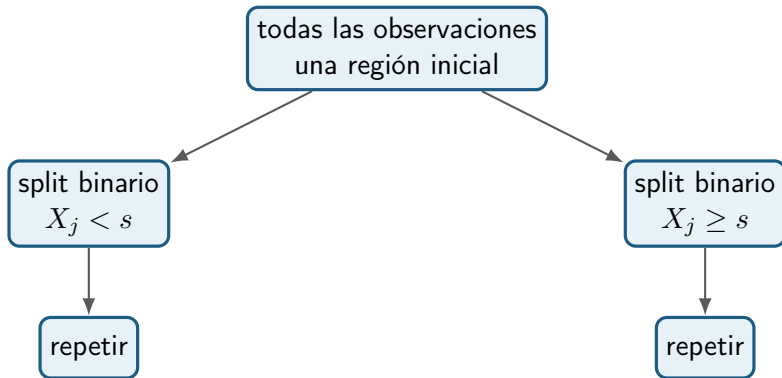
En regresión, cada región R_j predice con la media del outcome en el training set:

$$\hat{f}(x) = \sum_{j=1}^J \hat{c}_j \mathbb{I}\{x \in R_j\}, \quad \hat{c}_j = \frac{1}{|R_j|} \sum_{i: x_i \in R_j} y_i.$$

Recursive binary splitting

Enfoque

El procedimiento es de arriba hacia abajo y *greedy*: en cada paso busca la mejor división en ese punto, no la que necesariamente produciría el mejor árbol global.



Recursive binary splitting

El procedimiento top-down busca, en cada nodo, una variable X_k y un punto de corte s que minimicen el RSS de los dos nodos hijos:

$$\sum_{i: x_i \in R_1(k,s)} (y_i - \hat{c}_1)^2 + \sum_{i: x_i \in R_2(k,s)} (y_i - \hat{c}_2)^2.$$

Por qué es greedy

El algoritmo no enumera todas las particiones posibles. Evalúa divisiones locales y continúa recursivamente; por eso es computacionalmente viable pero sensible a perturbaciones de los datos.

Overfitting y poda

Prepoda

Definir hiperparámetros que detienen el crecimiento antes de máxima profundidad:

- profundidad máxima;
- número mínimo de casos por nodo;
- umbral mínimo de ganancia.

Postpoda

Construir un árbol grande y luego simplificar ramas desde abajo hacia arriba, reemplazando subárboles complicados por estructuras más simples.

Criterio práctico

Elegir complejidad mirando métricas en train/test con validación cruzada.

Poda: regularización de un árbol

Un árbol completamente crecido suele tener baja pérdida de entrenamiento y alta varianza. La poda reemplaza ese árbol por un subárbol más pequeño:

$$\hat{T}_\alpha = \arg \min_T \{R(T) + \alpha|T|\}.$$

Efecto de α

Si α aumenta, se penalizan más las hojas y el árbol se simplifica.

Selección

El valor de α se elige por validación cruzada o por desempeño en muestra de validación.

Poda como control de complejidad

La poda por complejidad de costo puede leerse como un problema penalizado:

$$C_{\alpha}(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{c}_m)^2 + \alpha |T|.$$

- El primer término premia ajuste dentro de hojas.
- El segundo término penaliza la cantidad de hojas.
- La validación cruzada selecciona un nivel de complejidad con menor error esperado fuera de muestra.

Cost-complexity pruning: penalización de árboles

Para un árbol de regresión:

$$\sum_{j=1}^{|T|} \sum_{i \in R_j} (y_i - y_{R_j})^2 + \alpha |T|.$$

Primer término

Suma de residuos cuadrados dentro de las regiones terminales R_j .

Segundo término

Penalización por número de nodos terminales:

$$\alpha |T|.$$

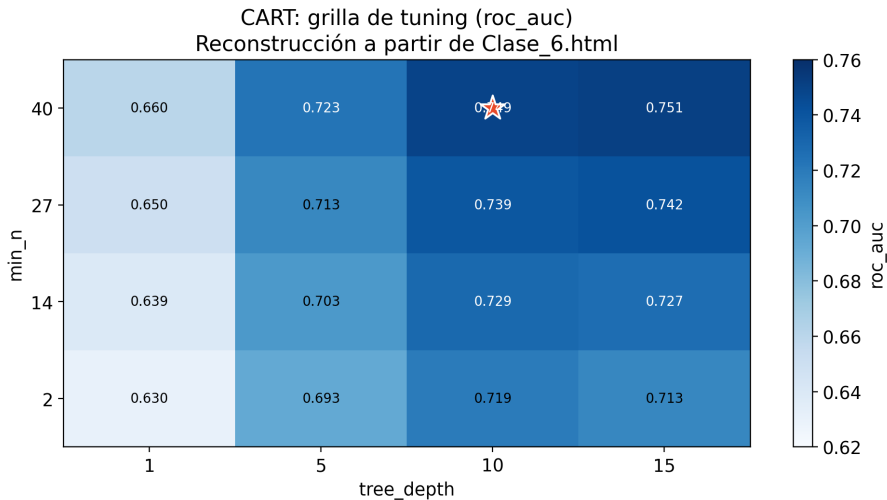
A mayor α , mayor presión hacia árboles pequeños.

Selección

El valor óptimo de α se identifica mediante validación cruzada.

Poda y validación: leer la grilla

La poda controla el tamaño efectivo del árbol. En la práctica se elige la complejidad por validación cruzada, buscando buen desempeño fuera de muestra y no sólo pureza dentro del entrenamiento.



CART con scikit-learn

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier

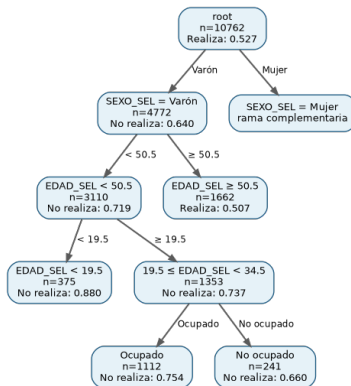
cart = DecisionTreeClassifier(random_state=123)

param_grid = {
    "ccp_alpha": [0.0, 0.001, 0.01, 0.05],
    "max_depth": [2, 4, 8, None],
    "min_samples_leaf": [5, 10, 25, 50],
}

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=123)

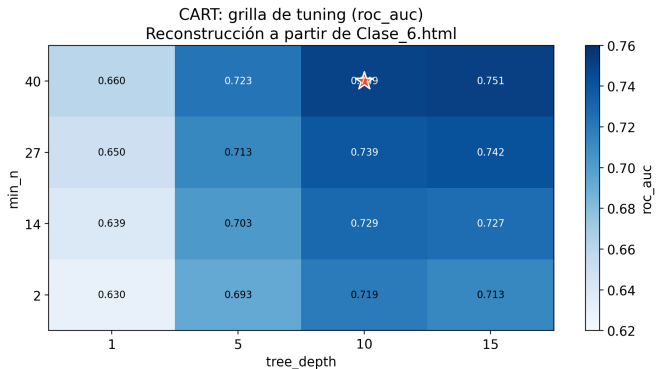
cart_rs = GridSearchCV(
    cart, param_grid=param_grid,
    scoring="roc_auc", cv=cv, n_jobs=-1
)
cart_rs.fit(X_train, y_train)
best_cart = cart_rs.best_estimator_
```

CART en ENUT 2021: árbol final del ejemplo



Primera partición por **SEXO_SEL**; luego cortes sucesivos en **EDAD_SEL**, condición de actividad y variables del hogar.

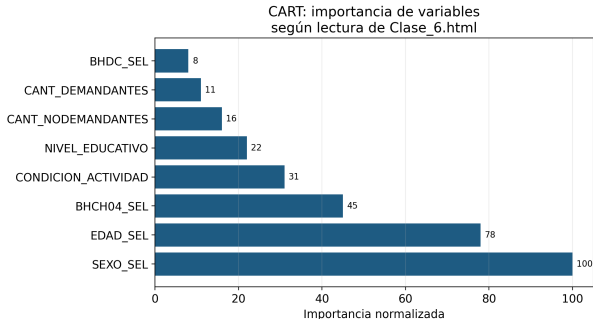
CART: grilla de tuning y selección



Mejor combinación en validación cruzada para `scoring=roc_auc`:

`ccp_alpha=1e-4, max_depth=10, min_samples_leaf=40.`

CART: qué significa importancia de variables



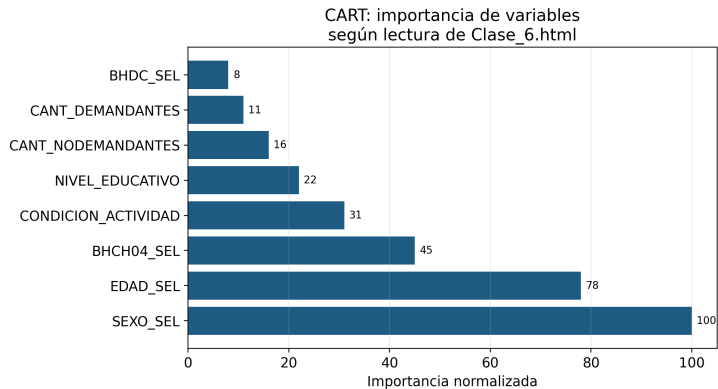
Definición operativa

Una variable es importante si aparece en cortes que reducen mucho la impureza, especialmente cerca de la raíz.

Cuidado

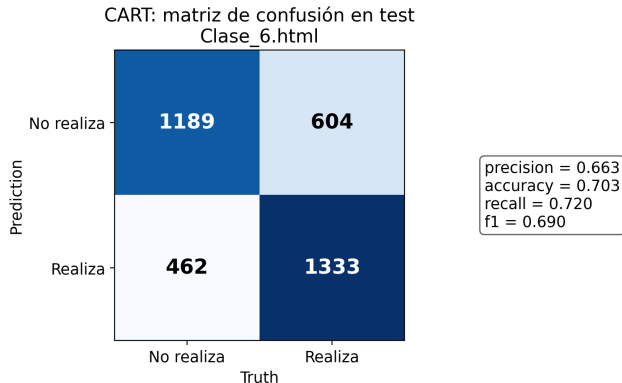
La importancia no es un coeficiente causal ni un efecto marginal. Resume utilidad predictiva dentro del árbol ajustado.

CART: importancia de variables



Variables dominantes: SEXO_SEL, EDAD_SEL y BHCH04_SEL.

CART: matriz de confusión en test



Salida exacta de test: precisión 0.663, exactitud 0.703, recall 0.720 y $F_1 = 0.690$.

CART: interpretable, pero inestable

Idea

Un árbol individual puede ser muy interpretable, pero también sensible a pequeñas perturbaciones de los datos.

- Un cambio pequeño puede modificar el primer split.
- Si cambia un split temprano, cambia buena parte de la estructura posterior.
- Esta inestabilidad motiva bagging y random forest.

Puente lógico

Pasamos de un árbol único a muchos árboles promediados para reducir varianza sin cambiar la lógica base de particiones.

Pausa activa 4: leer un árbol

Elegí una ruta raíz-hoja del árbol del ejemplo ENUT 2021.

- ➊ Escribí la regla completa en lenguaje natural.
- ➋ ¿Qué predicción hace esa hoja?
- ➌ ¿Qué ventaja interpretativa tiene esta regla?
- ➍ ¿Qué riesgo de inestabilidad tiene?

Bloque 6: ensambles e interpretación

Ensemble learning

Definición operativa

Ensamblar es combinar varios modelos de base para ampliar el espacio de hipótesis y mejorar performance.

Averaging

Construir estimadores independientes y promediar predicciones:

bagging, random forest, extra trees.

Reduce varianza.

Boosting

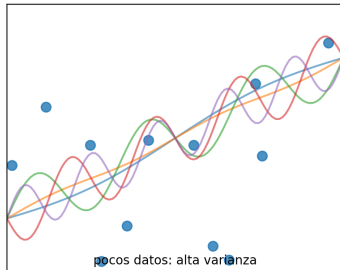
Construir estimadores secuencialmente para reducir sesgo:

AdaBoost, Gradient Boosting.

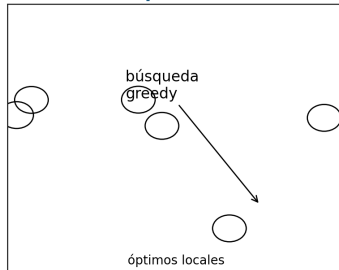
Ensamblas: problemas que motivan promediar

Por qué puede ayudar un ensemble

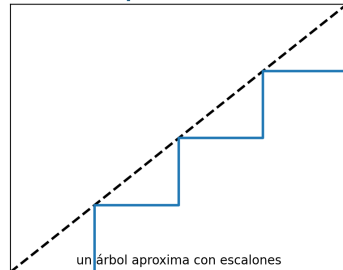
Estadístico



Computacional



Representación



Los ensambles ayudan cuando los modelos base son mejores que azar y suficientemente diversos: al combinar errores distintos, la predicción agregada puede aproximar mejor la función objetivo.

Condiciones para que un ensamble ayude

Dos condiciones hacen que un ensamble sea útil:

1. Capacidad predictiva

Los clasificadores base deben predecir mejor que el azar. En clasificación binaria, una referencia práctica es:

$$AUC > 0.5.$$

2. Diversidad

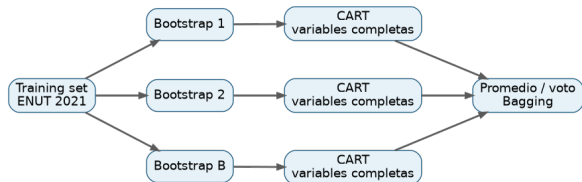
Los clasificadores base deben cometer errores distintos ante los mismos casos. Sin diversidad, el promedio no mejora sustantivamente la precisión.

Bagging: muchas muestras, muchos árboles

El bagging entrena árboles sobre remuestras bootstrap:

$$\mathcal{D}^{*(b)} = \{(x_i, y_i)\}_{i \in B_b}, \quad b = 1, \dots, B.$$

La predicción final se obtiene promediando probabilidades o votos.



Por qué funciona

Al promediar predictores inestables, se reduce la varianza de la predicción agregada.

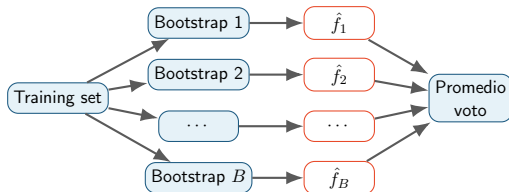
Bagging: bootstrap aggregation

Motivación

Los árboles simples tienen alta variabilidad: pequeños cambios en los datos pueden cambiar radicalmente el árbol final.

Idea

Aleatorizar los datos, entrenar muchos árboles y promediar sus predicciones.



Bagging: fórmula de predicción

Si se entrenan B modelos de base:

$$\hat{f}_1(x), \hat{f}_2(x), \dots, \hat{f}_B(x),$$

la predicción por promedio es:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

Clasificación

El promedio puede operar como voto mayoritario o promedio de probabilidades predichas.

Efecto esperado

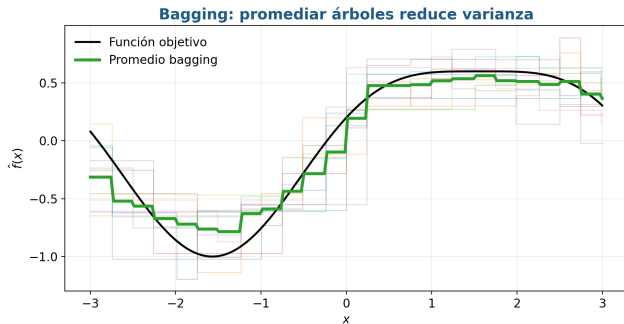
Al promediar modelos variables pero no idénticos, se reduce la varianza del error de generalización.

Promediar reduce varianza

Si los árboles individuales tienen varianza σ^2 y correlación promedio ρ , la varianza del promedio se aproxima por:

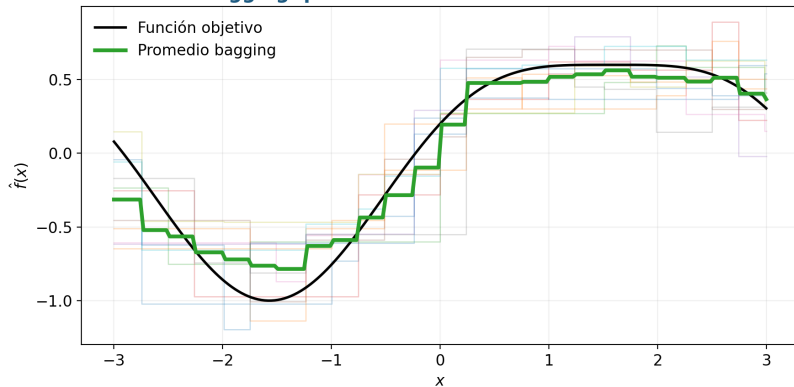
$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Aumentar B ayuda; reducir la correlación entre árboles ayuda todavía más.



Bagging: visualización de reducción de varianza

Bagging: promediar árboles reduce varianza



Bagging entrena B modelos sobre remuestras bootstrap y agrega:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

Bagging: error out-of-bag

Como cada árbol se entrena sobre una muestra bootstrap, algunas observaciones quedan fuera de esa muestra. Esas observaciones *out-of-bag* permiten estimar error sin separar un conjunto adicional de validación.

```
bag = BaggingClassifier(  
    estimator=DecisionTreeClassifier(random_state=123),  
    n_estimators=500,  
    bootstrap=True,  
    oob_score=True,  
    random_state=123,  
    n_jobs=-1,  
)
```

Lectura

OOB es una evaluación interna útil del bagging, pero el test set sigue siendo la estimación final una vez cerrado el proceso de selección.

Bagging con scikit-learn

```
from sklearn.ensemble import BaggingClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

base_tree = DecisionTreeClassifier(random_state=123)
bag = BaggingClassifier(
    estimator=base_tree, n_estimators=500,
    bootstrap=True, oob_score=True, random_state=123,
    n_jobs=-1
)

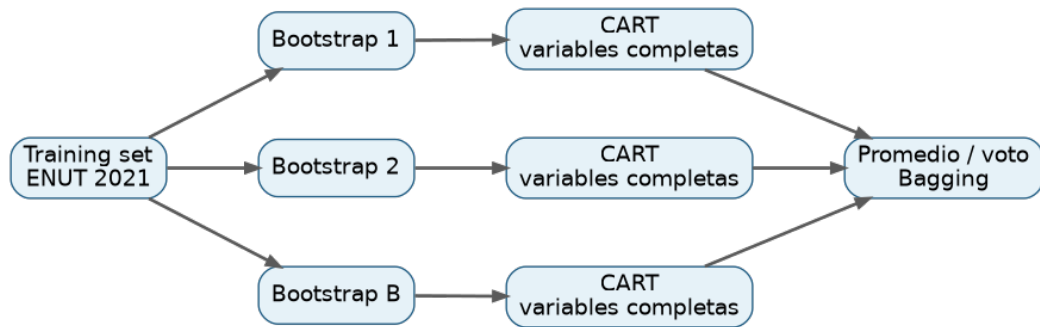
param_grid = {
    "estimator__max_depth": [None, 6, 10],
    "estimator__min_samples_leaf": [1, 5, 20],
}

bag_rs = GridSearchCV(bag, param_grid, scoring="roc_auc", cv=cv)
best_bag = bag_rs.fit(X_train, y_train).best_estimator_
```

Detalle didáctico

Como el bagging promedia muchos árboles, preocupa menos que cada árbol individual sea profundo o complejo.

Bagging: flujo del modelo implementado



Se agregan árboles CART entrenados sobre remuestreos bootstrap: `DecisionTreeClassifier` dentro de `BaggingClassifier`.

Bagging: especificación en scikit-learn

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

base_tree = DecisionTreeClassifier(random_state=123)

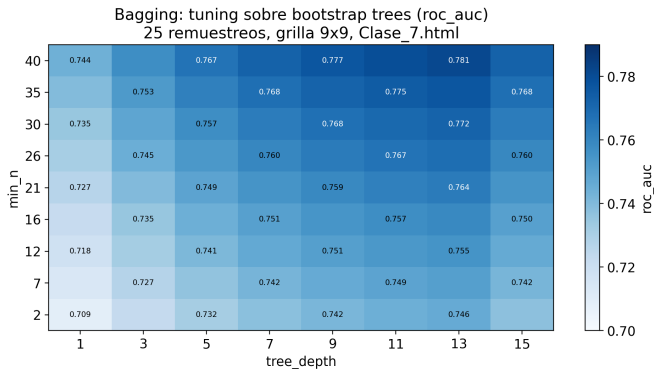
bag = BaggingClassifier(
    estimator=base_tree,
    n_estimators=500,
    bootstrap=True,
    oob_score=True,
    random_state=123,
    n_jobs=-1,
)

param_grid = {
    "estimator__max_depth": [None, 5, 10],
    "estimator__min_samples_leaf": [1, 5, 20],
}
```

Interpretación

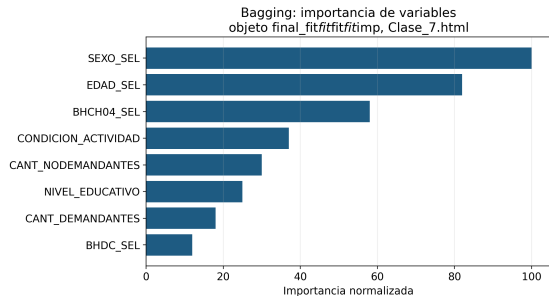
A diferencia del árbol simple, el bagging puede usar árboles de base más profundos, porque el promedio de remuestreos reduce varianza

Bagging: grilla de tuning



Grilla para estimator__max_depth y estimator__min_samples_leaf; selección por scoring=roc_auc.

Bagging: importancia de variables en Python



```
from sklearn.inspection import permutation_importance

result = permutation_importance(
    best_bag, X_test, y_test,
    scoring="roc_auc",
    n_repeats=20,
    random_state=123,
    n_jobs=-1,
)
```

Lectura

La importancia por permutación mide cuánto cae la métrica fuera de muestra al romper la asociación entre una variable y la respuesta.

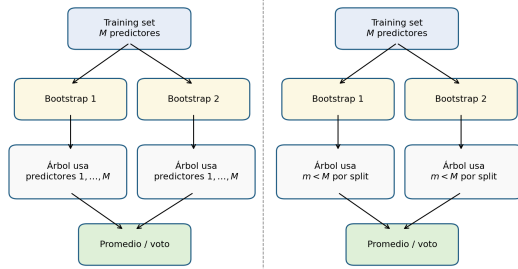
Random forest: bagging con decorrelación

Random forest conserva la idea bootstrap de bagging, pero en cada corte sólo permite competir a un subconjunto aleatorio de predictores.

Intuición

Si todos los árboles ven siempre los mismos predictores fuertes, tenderán a parecerse. Restringir predictores por nodo disminuye su correlación.

Bagging vs Random Forest: dos fuentes de aleatoriedad



Aleatoriza registros; árboles pueden correlacionarse si hay predictores dominantes; aleatoriza registros y predictores; reduce correlación entre árboles base.

Random Forest

Random forest es muy similar al bagging de árboles, con una diferencia decisiva:

Doble aleatorización

- 1 Aleatoriedad sobre registros: remuestras bootstrap.
- 2 Aleatoriedad sobre predictores: en cada split se considera un subconjunto de variables.

Por qué ayuda

Si una variable es muy fuerte, bagging tiende a seleccionarla en muchos árboles, generando árboles correlacionados. Al seleccionar subconjuntos aleatorios de variables en cada división, se reduce esa correlación.

Random Forest: de bagging a decorrelación



En cada split se evalúa sólo un subconjunto aleatorio de predictores: esto decorrelaciona árboles.

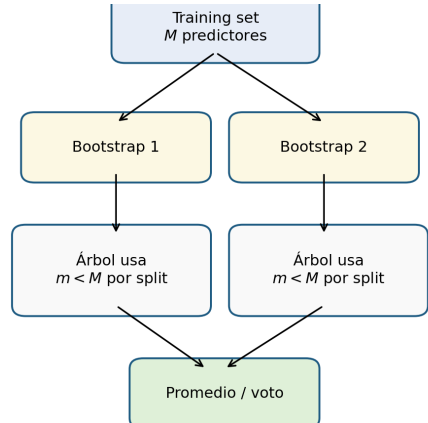
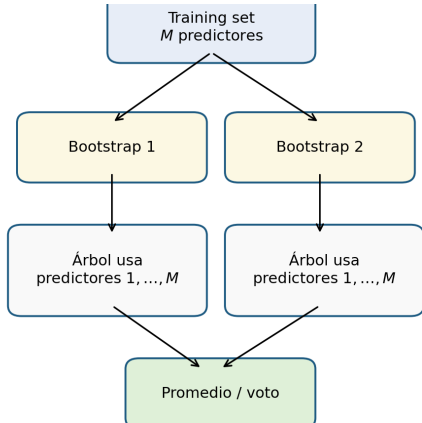
Bagging vs. Random Forest

Método	Aleatoriza registros	Aleatoriza predictores
Árbol CART	No	No
Bagging	Sí: bootstrap	No: usa los M predictores
Random Forest	Sí: bootstrap	Sí: subconjunto en cada split
Extra Randomized Trees	Sí	Sí, y además splits aleatorios

Regla práctica mencionada

Para clasificación con p variables suele usarse $\sqrt{p}$ variables por división; para regresión, $p/3$, aunque puede tunearse como hiperparámetro.

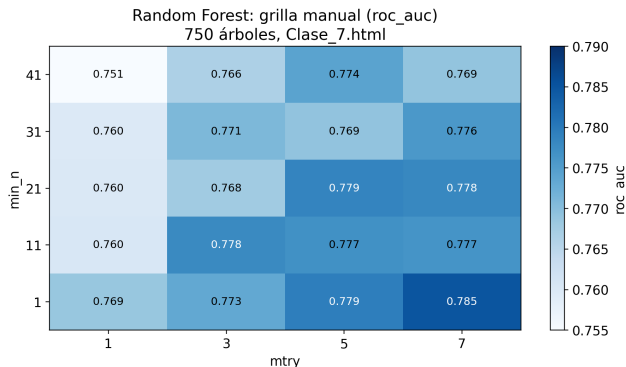
Bagging vs Random Forest



Qué se tunea en un bosque aleatorio

Los hiperparámetros más importantes son:

- número de árboles;
- cantidad de predictores candidatos por corte;
- tamaño mínimo de nodo;
- profundidad efectiva inducida por las restricciones anteriores.



Random Forest: hiperparámetro de predictores

En cada split, random forest selecciona aleatoriamente sólo m de los p predictores disponibles. Las reglas heurísticas presentadas en clase son:

$$m \approx \sqrt{p} \quad \text{para clasificación,} \quad m \approx p/3 \quad \text{para regresión.}$$

Efecto estadístico

La selección aleatoria de predictores reduce la correlación entre árboles, especialmente cuando una variable muy fuerte tendería a dominar todos los splits en bagging.

Random Forest con scikit-learn

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(
    n_estimators=750,
    bootstrap=True,
    random_state=123,
    n_jobs=-1,
)

param_grid = {
    "max_features": [1, 3, 5, "sqrt"],
    "min_samples_leaf": [1, 10, 20, 50],
}

rf_rs = GridSearchCV(rf, param_grid, scoring="roc_auc", cv=cv)
rf_rs.fit(X_train, y_train)
best_rf = rf_rs.best_estimator_
```


Random Forest: especificación en scikit-learn

```
from sklearn.ensemble import RandomForestClassifier

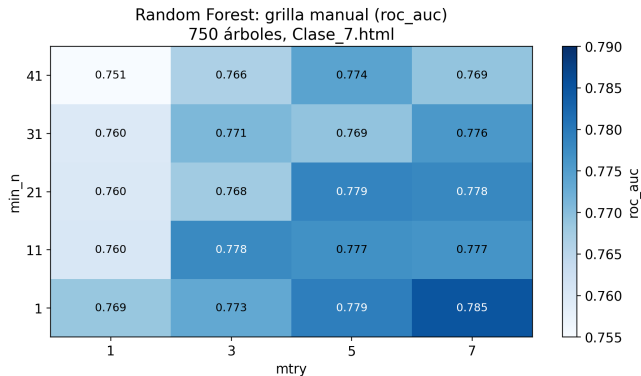
rf = RandomForestClassifier(
    n_estimators=750,
    max_features="sqrt",
    min_samples_leaf=1,
    bootstrap=True,
    random_state=123,
    n_jobs=-1,
)

param_grid = {
    "max_features": [1, 3, 5, "sqrt"],
    "min_samples_leaf": [1, 10, 20, 50],
}
```

Diferencia clave

`max_features` controla cuántos predictores compiten en cada partición; por eso random forest no sólo remuestrea filas, sino también columnas.

Random Forest: grilla manual de hiperparámetros



Grilla manual: `max_features` $\times$ `min_samples_leaf`; selección por `scoring=roc_auc`.

Interpretabilidad: tres niveles

Coeficientes	Reglas	Post-hoc
En regresión, un parámetro resume una asociación parcial bajo una forma funcional.	En CART, una ruta raíz-hoja define una regla condicional interpretable.	En random forest, se interpretan patrones agregados: importancia, PDP, ICE, ALE.

Idea central

A mayor flexibilidad predictiva, mayor necesidad de herramientas de interpretación externa.

Importancia por impureza vs. importancia por permutación

Impureza

Suma la reducción de impureza atribuida a cada variable a lo largo de los árboles.

$$VI_j^{imp} = \sum_{\text{splits en } j} \Delta I.$$

Puede favorecer variables continuas o con muchas categorías.

Permutación

Mide cuánto cae la performance al romper la relación entre X_j e Y .

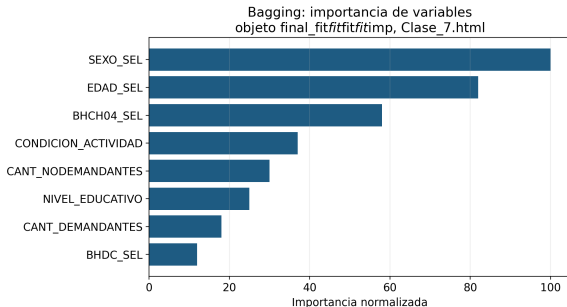
$$VI_j^{perm} = \text{Score}(\hat{f}) - \text{Score}(\hat{f}; X_j^\pi).$$

Con predictores correlacionados puede subestimar variables sustituibles.

Advertencia causal

La importancia de variables no mide efecto causal: mide contribución predictiva bajo este modelo y estos datos.

Importancia de variables en ensambles



Lectura

En un ensamble, la importancia resume contribuciones promedio de muchos árboles. Por eso es más estable que en un único CART, pero menos local y menos transparente.

Pregunta correcta

No es “cuánto cambia Y si cambia X_j ”, ni “cuál es el efecto causal de X_j ”. Es “cuánto ayuda X_j a predecir en este algoritmo”.

PDP: efecto promedio manteniendo el modelo fijo

Para una variable x_s , el Partial Dependence Plot estima

$$\hat{f}_s(z) = \frac{1}{n} \sum_{i=1}^n \hat{f}(z, x_{i,-s}).$$

Interpretación

Muestra cómo cambia la predicción promedio si fijamos $x_s = z$ para todas las observaciones, manteniendo los demás predictores como fueron observados.

Limitación

Si x_s está fuertemente correlacionado con otros predictores, el PDP puede evaluar combinaciones poco plausibles.

ICE: heterogeneidad de efectos predictivos

Un gráfico ICE calcula una curva por observación:

$$\hat{f}_s^{(i)}(z) = \hat{f}(z, x_{i,-s}).$$

El PDP es el promedio de esas curvas:

$$\hat{f}_s(z) = \frac{1}{n} \sum_{i=1}^n \hat{f}_s^{(i)}(z).$$

Lectura

Si las curvas ICE son paralelas, el efecto predictivo es relativamente homogéneo. Si se cruzan o divergen, hay interacción o heterogeneidad.

Interpretabilidad post-hoc en Python

```
from sklearn.inspection import permutation_importance, PartialDependenceDisplay

result = permutation_importance(
    best_model, X_test, y_test,
    n_repeats=20,
    scoring="roc_auc",
    random_state=123
)

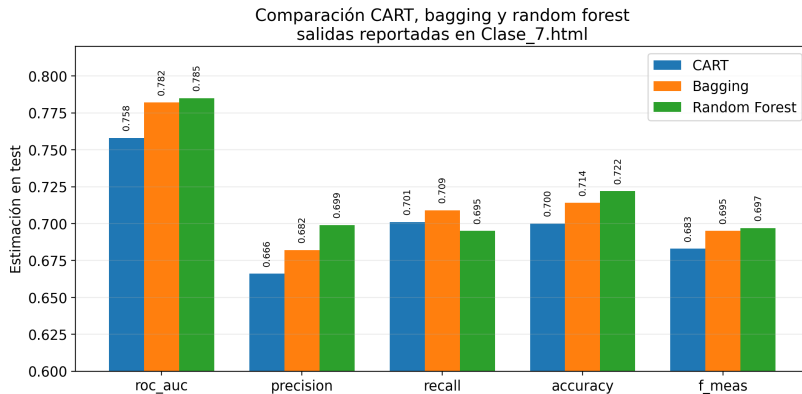
PartialDependenceDisplay.from_estimator(
    best_model,
    X_test,
    features=["edad", "educacion"]
)
```

Buenas prácticas

Calcular interpretación post-hoc sobre datos separados de entrenamiento ayuda a no confundir ajuste interno con comportamiento fuera de muestra.

Cierre: comparación y guía de decisión

Comparación final: CART, Bagging y Random Forest



RF obtiene la mayor exactitud y AUC; bagging mejora al CART simple por promediar árboles.

Evaluación comparada

El flujo de evaluación es común a CART, bagging y random forest:

entrenar en training set → tunear con cross-validation → evaluar en test set.

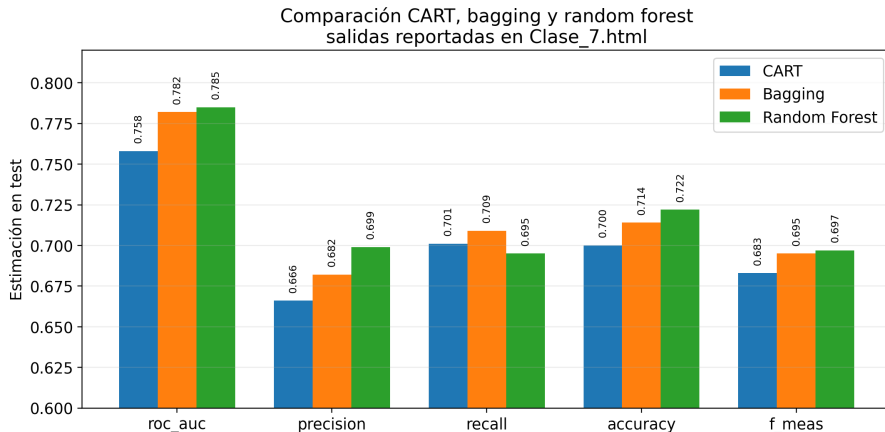
Métricas

precision, recall, accuracy, F_1 , ROC/AUC .

Pregunta final

No se elige el modelo por nombre sino por desempeño empírico y adecuación interpretativa al problema sustantivo.

Comparación analítica: CART, bagging y random forest



Lectura

La mejora de un ensamble puede ser real pero marginal. La elección final debe balancear desempeño, estabilidad, costo computacional e interpretabilidad.

Qué se gana y qué se pierde

Modelo	Sesgo	Varianza	Interpretabilidad
Regresión lineal/logística	medio	baja	alta
CART	baja-media	alta	alta
Bagging	similar a árbol profundo	menor	media-baja
Random forest	baja	menor	baja-media

Síntesis

Los ensambles no eliminan el problema de elegir modelo: lo desplazan hacia la evaluación, la explicación y el control del error fuera de muestra.

Cierre: una misma lógica de control de complejidad

Aunque los modelos parecen distintos, todos enfrentan la misma tensión:

flexibilidad predictiva $\longleftrightarrow$ capacidad de generalización.

Familia	Complejidad	Control
Regresión penalizada	tamaño de coeficientes	λ, α
Logística penalizada	log-verosimilitud regularizada	C, λ, α
CART	número de hojas/profundidad	poda, max_depth
Bagging	árboles promediados	B , profundidad
Random forest	árboles decorrelacionados	max_features, B

Extensión opcional: Extra Trees y boosting

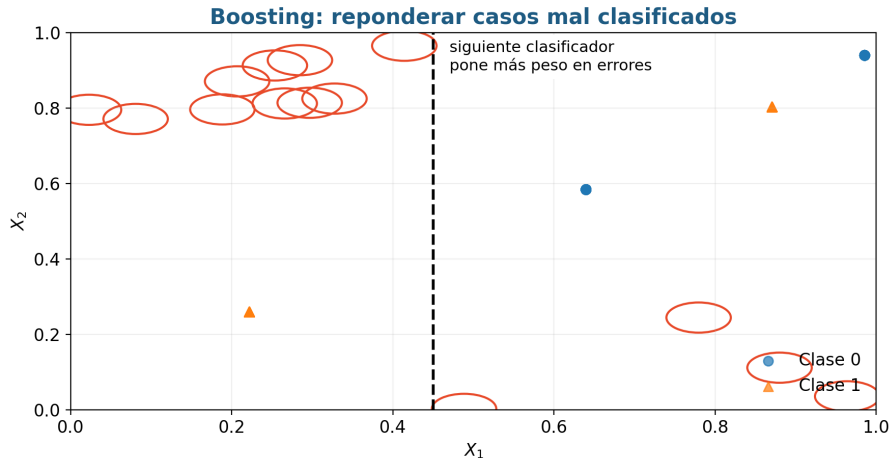
Hasta aquí la clase se concentró en averaging. Como extensión conceptual, los ensambles se pueden ordenar en dos familias:

Averaging: bagging, random forest, extra trees

Boosting: AdaBoost, gradient boosting, XGBoost.

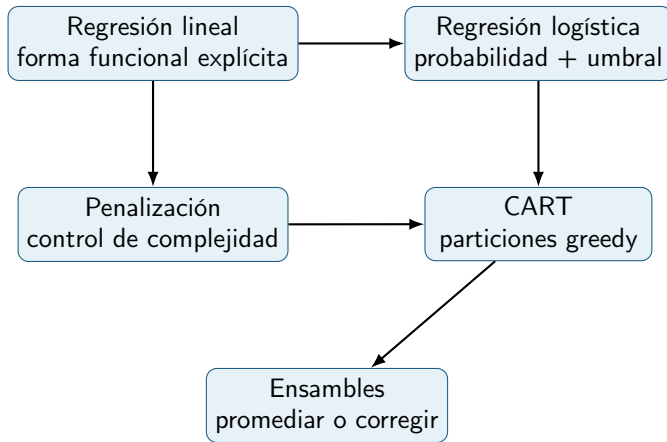
- Extra Trees agrega mayor aleatoriedad: además de submuestrear predictores, puede aleatorizar puntos de corte.
- Boosting construye modelos secuencialmente, intentando corregir errores del modelo anterior.
- La intuición pasa de reducir varianza por promedio a reducir sesgo mediante actualización iterativa.

Boosting: reponderar errores



En AdaBoost, las observaciones mal clasificadas reciben más peso en la siguiente iteración, y el ensamble final combina clasificadores con voto ponderado según su error.

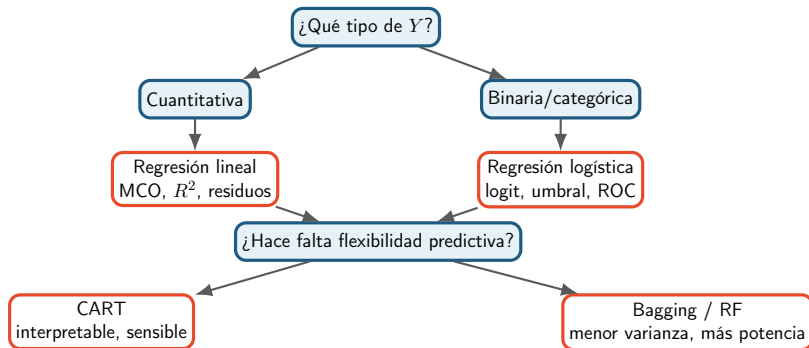
Mapa ampliado de modelos



Lectura final

La progresión no reemplaza modelos simples por modelos complejos; enseña cuándo el costo de complejidad se justifica por error predictivo, estabilidad o adecuación al problema.

Mapa de decisiones



Guía de elección del modelo

Priorizar interpretación paramétrica

Regresión lineal o logística: coeficientes, hipótesis sustantivas, efectos ajustados.

Priorizar desempeño predictivo

Bagging o random forest: menor varianza y mejor generalización, con menor transparencia global.

Priorizar reglas simples

CART: lectura directa de particiones, pero con riesgo de inestabilidad.

Priorizar control de complejidad

Regularización, selección, poda y validación cruzada controlan complejidad, pero operan de maneras distintas.

Pausa activa final: elegir modelo según objetivo

Mismo problema, tres modelos candidatos:

- 1 logística penalizada;
- 2 CART podado;
- 3 random forest.

Elegí uno si el objetivo principal es:

- explicación simple ante una audiencia no técnica;
- máxima precisión predictiva fuera de muestra;
- equilibrio entre estabilidad, interpretación y performance.

Cierre lógico

No se elige un modelo por prestigio técnico, sino por la relación entre pregunta sustantiva, métrica, costo de error e interpretabilidad.

Síntesis conceptual

- 1 La regresión lineal modela una media condicional mediante una recta o hiperplano y se ajusta por MCO.
- 2 La regresión logística modela probabilidades binarias usando el enlace logit; la clase predicha aparece recién al elegir un umbral.
- 3 La regularización achica coeficientes; AIC/BIC comparan modelos; la poda simplifica árboles; la validación cruzada elige hiperparámetros.
- 4 CART particiona recursivamente el espacio predictor, con alta interpretabilidad y riesgo de inestabilidad.
- 5 Bagging reduce varianza promediando árboles entrenados sobre remuestras bootstrap.
- 6 Random forest agrega aleatorización de predictores mediante `max_features` para reducir correlación entre árboles.
- 7 En ciencias sociales, desempeño predictivo, interpretabilidad y causalidad son criterios distintos: no deben confundirse.